

Universidad de Guadalajara
Centro Universitario de Los Valles



Calidad de Software

Pruebas de Software en Trabajo de Tesis: Proyecto Integrador.

Profesora:

Mtra. Sonia López Ruiz

Alumno:

Ing. Josue Ernesto Galindo Morales

Carretera Ameca-Guadalajara, Jalisco, México

A 13 de mayo de 2024

Índice.

1.	Definir el problema o caso de estudio que será analizado en cuanto a la calidad.....	4
2.	Definir el objetivo general.....	5
2.1	Objetivo general.....	5
2.2.	Objetivos específicos.....	5
3.	Describir los requerimientos generales del software.....	6
4.	Describir los criterios y métricas de calidad que serán aplicados al software.....	7
4.1.	Características de la calidad.....	8
4.2.	Características de la calidad interna.....	8
4.3.	Características de la calidad externas.....	9
4.4.	Características de la calidad en producción.....	9
4.5.	Selección de las características de calidad.....	10
4.6.	Modelo de indicadores y métricas.....	12
5.	Describir la aplicación de las métricas seleccionadas.....	14
5.1.	Entorno de pruebas.....	14
5.2.	Adecuación funcional.....	15
5.3.	Ejecución de las pruebas.....	18
5.3.1.	Comentarios por línea de código.....	18
5.3.2.	Modularidad.....	19
5.3.3	Consistencia en tiempos de respuesta.....	21
5.3.4.	Porcentaje de requerimientos funcionales implementados.....	22
5.3.5.	Porcentaje de pruebas de requerimientos funcionales exitosas.....	23
5.3.6.	Eficacia de los controles de integridad.....	24
5.3.7.	Número de brechas de confidencialidad.....	25
5.3.8.	Tasa de fallos de autenticación.....	27
5.3.9.	Uso de memoria.....	28
5.3.10.	Tiempo de respuesta.....	29
5.3.11.	Número de plataformas soportadas.....	33
5.3.12.	Tiempo promedio de instalación.....	35
6.	Describir la matriz de resultados.....	36

7.	Resultados de la evaluación.....	39
8.	Conclusión.	40
	Referencias utilizadas.....	42

1. Definir el problema o caso de estudio que será analizado en cuanto a la calidad.

Como ya se indica en el título, este proyecto consiste en analizar a detalle todos los elementos que conforman la calidad de este proyecto de software. Durante este trabajo se identificarán cuáles son esos elementos, subelementos, métricas y ponderaciones que nos permitirán generar una calificación, y se analizarán dichos resultados para llegar a una conclusión; ya sea positiva o negativa sobre el software que estamos desarrollando.

En el CUVALLES los académicos y estudiantes enfrentan desafíos significativos en la comunicación y el intercambio de información debido a la falta de un sistema integrado que permita la transmisión eficiente de video, audio, mensajes y archivos.

Los métodos actuales de comunicación, que a menudo involucran una mezcla de plataformas dispersas y no integradas, han llevado a problemas como:

- a. **Ineficiencia en la comunicación:** Los académicos experimentan demoras y dificultados en la comunicación debido a la falta de un sistema unificado que soporte diversos tipos de datos y formatos de comunicación en tiempo real.
- b. **Seguridad y privacidad:** La utilización de múltiples plataformas para la comunicación y el intercambio de archivos sensibles no siempre cumple con los estándares de seguridad necesarios, poniendo en riesgo la privacidad de la información académica y personal.

Este panorama conduce a la necesidad de desarrollar y evaluar un sistema de comunicación y mensajería que no solo aborde estos problemas mediante el uso eficiente de los protocolos TCP/UPD en la LAN del CUVALLES, sino que también sea evaluado rigurosamente en términos de calidad para garantizar su funcionalidad, seguridad, accesibilidad y eficiencia.

2. Definir el objetivo general.

2.1 Objetivo general.

El objetivo de realizar este análisis es el de evaluar la calidad del sistema de comunicación y mensajería para asegurar su eficacia, eficiencia, seguridad y usabilidad dentro de la red LAN de CUVALLES. Mediante la selección de 3 métricas de calidad, analizando los requerimientos funcionales, y ponderando escalas de medición y comparación obtendremos un resultado numérico de nuestra evaluación de calidad. De esta manera tomarla como retroalimentación y aseguraremos de tomar las medidas adecuadas para mejorar el sistema.

2.2. Objetivos específicos.

- **Analizar y evaluar la calidad del software** mediante la aplicación de métricas específicas para cada característica definida (como la funcionalidad, usabilidad, rendimiento, seguridad y mantenibilidad), identificando áreas clave para mejoras.
- **Ponderar y clasificar las métricas de calidad** para determinar su relevancia en relación con los objetivos del proyecto, permitiendo una focalización efectiva en los aspectos más críticos que impactan la experiencia del usuario y la eficacia operativa.
- **Desarrollar y mantener un protocolo de pruebas continuo** que incluya pruebas automatizadas y manuales para evaluar regularmente la funcionalidad y la estabilidad del sistema, ajustando y recalibrando las métricas según sea necesario.
- **Establecer un sistema de seguimiento y revisión de la seguridad** del software que incluya pruebas de vulnerabilidad y la implementación de correcciones en tiempo real para proteger la integridad y privacidad de los datos de los usuarios.
- **Documentar y reportar los avances y resultados de las evaluaciones de calidad** a los stakeholders y al equipo de desarrollo (yo), proporcionando transparencia y fomentando una filosofía de mejora continua dentro del proyecto.
- **Establecer un marco de trabajo para la incorporación continua de nuevas tecnologías y prácticas emergentes** en el sistema, asegurando que el software permanece actualizado y competitivo en un entorno tecnológico en rápida evolución.



3. Describir los requerimientos generales del software.

Los requerimientos son la base de la que partir si debemos iniciar un proyecto de software. Dicho de mejor manera: **son la base de cualquier proyecto**. Ya que de estas se modela, se desarrolla y se toman decisiones durante el proyecto. Encontramos los RF (Requerimientos Funcionales) que otorgan toda la funcionalidad del proyecto (las funciones que satisfacen la necesidad planteada); y los RNF (Requerimientos No Funcionales) que nos otorgan el cómo se logrará el proyecto (customizaciones, metodologías, tecnologías, etc...).

Los requerimientos funcionales que se solicitan para este proyecto son los que se muestran en la *tabla 1*.

Requerimientos funcionales	
RF1	Sistema responsivo cel/pc.
RF2	Los usuarios deben iniciar sesión.
RF3	Mostrar "online/offline" de los usuarios con el circulito rojo o verde.
RF4	Mostrar, y permitir modificar, si se desan activar o no las notificaciones con sonido.
RF5	Búsqueda de chat deseado.
RF6	En chat, se pueden enviar mensajes de texto e imágenes.
RF7	El usuario puede crear grupo y asignar roles: lector, regular, admin.
RF8	Mostrar información del grupo o usuario con el que se está hablando.
RF9	Cambiar foto de perfil.
RF10	Cambiar nombre que se muestra.
RF11	Cambiar estatus.
RF12	Guardar o cancelar cambios.
RF13	Cifrar toda la información de extremo a extremo.
RF14	Eliminar información de base de datos con antigüedad mayor a 15 días.
RF15	Aplicación debe funcionar enteramente con recursos en LAN.

Tabla 1

Y los requerimientos no funcionales que se presentan para este proyecto se muestran en la *tabla 2*.

Requerimientos No Funcionales	
RNF1	Implementación de sockets y protocolos TCP/UDP.
RNF2	Desarrollar e implementar una base de datos MySQL.
RNF3	Desarrollo de servidor Python FastAPI.
RNF4	Implementacion de JavaScript (vanilla o framework).
RNF5	Implementar llamadas y videollamadas mediante conexión P2P usando librería WebRTC.
RNF6	Mensajes de multimedia o audio serán almacenados en servidor con dirección estática.
RNF7	Mensajes de texto serán almacenados en base de datos.

Tabla 2

Como se puede observar en las tablas presentadas, el proyecto incluye un total de 15 requerimientos funcionales y 7 no funcionales. Para el propósito de este análisis, nos centraremos principalmente en los requerimientos funcionales, dado que son el núcleo de este proyecto y esenciales para su éxito.

4. Describir los criterios y métricas de calidad que serán aplicados al software.

Las series de normas ISO/IEC 25000, conocida como SQuaRE (Software Product Quality Requirements and Evaluation), ofrece un marco integral para evaluar y definir la calidad del software. Este con junto de estándares internacionales, desarrollado por la Organización Internacional de Normalización (ISO) y la Comisión Electrotécnica Internacional (IEC), es esencial para desarrolladores, gestores de calidad y evaluadores de software que buscan establecer criterios claros y sistemáticos para la medición de la calidad de los productos de software.

Dentro de la serie ISO/IEC 25000, varios estándares se concentran específicamente en las métricas de calidad. Por ejemplo, la norma ISO/IEC 25010:2011 define un modelo de calidad que identifica características clave como la funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad, cada una subdividida en subcaracterísticas que detallan aspectos más específicos de la calidad del software. A su vez, la norma ISO/IEC 25023:2016 proporciona un conjunto detallado de métricas para evaluar estas características y subcaracterísticas, lo que permite a los profesionales de software cuantificar de manera efectiva la calidad de un producto de software en términos concretos y objetivos.

Implementar estas métricas no solo facilita una evaluación rigurosa del software, sino que también ayuda a los equipos de desarrollo a identificar áreas de mejora y garantizar que los productos finales cumplen con los requisitos establecidos por los usuarios y stakeholders. Además, la norma ISO/IEC 25000 se enfoca en la calidad en uso, esencial para medir cómo el software satisface las necesidades en condiciones reales de operación, cubriendo aspectos como la efectividad, eficiencia, satisfacción y seguridad del usuario.

4.1. Características de la calidad.

En la evaluación de la calidad implementaremos 3 métricas por cada categoría de calidad: interna, externa y de producción. De estos grupos, las métricas de productos son las más generalizadas en los procesos de evaluación de software. Estas métricas consideran el desarrollo del producto en sí mismo, por lo cual son utilizadas en la fase final del proceso, es decir, cuando el producto está listo para su uso.

4.2. Características de la calidad interna.

Estas se refieren a las medidas que se aplican a los productos de software en un entorno no operativo, es decir, durante las fases de diseño y codificación antes de que el software sea ejecutado. Estas métricas se centran en los atributos internos del software que son medibles independiente mente de su comportamiento en tiempo de ejecución.

Para esta sección, las métricas seleccionadas para evaluar la calidad son:

- Mantenibilidad.
- Comentarios por línea de código.
- Escalabilidad.
- Modularidad.
- Consistencia en tiempo de respuesta.
- Porcentaje de requerimientos cubiertos.
- Porcentaje de Requerimientos funcionales implementados.
- Porcentaje de pruebas de requerimientos funcionales exitosas.

4.3. Características de la calidad externas.

Las métricas externas evalúan el comportamiento del sistema de software durante o después de su ejecución en condiciones de operación normales. Se aplican en cuando el software está en su entorno de ejecución y se interactúan con él como lo harían los usuarios finales. Estas métricas son cruciales para evaluar la calidad desde la perspectiva del usuario.

Las métricas seleccionadas para evaluar la calidad son:

- Seguridad.
- Eficacia de los controles de integridad.
- Número de brechas de confidencialidad.
- Tasa de fallos de autenticación.
- Eficiencia.
- Uso de memoria.
- Tiempo de respuesta.
- Portabilidad.
- Número de plataformas soportadas.
- Tiempo medio de instalación.

4.4. Características de la calidad en producción.

En estas nos centramos en el análisis del software en el contexto real de uso por parte de los usuarios finales. Estas métricas son altamente valiosas para entender cómo el software satisface las necesidades en condiciones reales y cuál es su impacto en el entorno operativo.

Las métricas que se escogieron para evaluar la calidad son:

- Rendimiento.
- Consistencia en tiempo de respuesta.
- Transacciones por segundo.
- Solicitudes por segundo.
- Usabilidad.
- Número de errores cometidos por nuevos usuarios.
- Tiempo promedio para completar tareas.
- Fiabilidad.

- Frecuencia de fallos.
- Tiempo promedio para recuperarse de fallos.

4.5. Selección de las características de calidad.

La calidad del software es un aspecto crítico que determina su eficacia, eficiencia y aceptación entre los usuarios. Por ello, es imprescindible definir claramente los criterios y medidas de calidad que se aplicarán para evaluar el sistema de comunicación y mensajería desarrollado para los académicos del CUVALLES. Estos criterios se han establecido con base en las métricas anteriormente descritas.

En la *figura 3* se muestra la ponderación del nivel de importancia de cada una de las características de calidad.

Nivel de importancia	Simbología	Porcentaje referencial	Significado
Crítico	C	(91% - 100%)	Fundamentalmente relevante en la medición.
Alto	A	(61% - 90%)	El nivel de importancia es alto y debe ser tomado en cuenta.
Moderado	M	(31% - 60%)	La métrica o submétrica puede o no ser medida, dependiendo del criterio del evaluador.
Bajo	B	(1% - 30%)	Tiene muy poca relevancia como para ser medida.
Nulo	N	(0%)	Este valor se dará a la característica que no se pueda medir dependiendo de diversos factores.

Tabla 3

Características de calidad interna, externa y en uso		
Interna	Mantenibilidad	A
	Escalabilidad	M
	Porcentaje de requerimientos cubiertos	C
Externa	Seguridad	C
	Eficiencia	M
	Portabilidad	B
En producción	Rendimiento	B
	Usabilidad	M
	Fiabilidad	C

Tabla 4

En la tabla 4 se muestra el proceso de evaluación de la calidad del software CUVALLS Messenger se estructura en torno a un conjunto riguroso de métricas y submétricas divididas en tres categorías principales: calidad interna, externa y en uso. Cada una de estas métricas ha sido calibrada con un grado de importancia, que oscila entre 'Bajo' (B), 'Medio' (M), y 'Alto' (A), reflejando su relevancia para la operatividad y la percepción del usuario del software.

Características o subcaracterísticas de calidad interna, externa y en uso y su descripción			
Tipo de calidad	Característica	Subcaracterística	Grado de importancia
Interna	Mantenibilidad	Comentarios por línea de código	A
	Escalabilidad	Modularidad	M
		Consistencia en tiempos de respuesta	M
	Porcentaje de requerimientos cubiertos	Porcentaje de requerimientos funcionales implementados	B
		Porcentaje de prueba de requerimientos funcionales exitosas	C
Externa	Seguridad	Eficacia de los controles de integridad	A
		Número de brechas de confidencialidad	M
		Tasa de fallos de autenticación	C
	Eficiencia	Uso de memoria	M
		Tiempo de respuesta	M
	Portabilidad	Número de plataformas soportadas	M
		Tiempo medio de instalación	B
En producción	Rendimiento	Consistencia en tiempo de respuesta	A
		Transacciones por segundo	M
		Solicitudes por segundo	M
	Usabilidad	Número de errores comentidos por nuevos usuarios	M
		Tiempo promedio para completar tareas	A
	Fiabilidad	Frecuencia de fallos	C
		Tiempo promedio para recuperarse de fallos	A

Tabla 5

La matriz prioriza la 'Fiabilidad', la 'Mantenibilidad' y la 'Seguridad' como aspectos de alta importancia, señalando su papel crítico en la confiabilidad y sostenibilidad a largo plazo del CUVALLS Messenger. Por otro lado, 'Portabilidad' y 'Rendimiento' se consideran de menor importancia en el contexto actual, aunque esto podría cambiar conforme el proyecto evoluciona y nuevas demandas surjan.

Características y subcaracterísticas de calidad interna, externa y en uso y su descripción			
Tipo de calidad	Característica	Subcaracterística	Descripción
Interna	Mantenibilidad	Comentarios por línea de código	Porcentaje entren el número de líneas de comentarios y el número total de líneas de código.
	Escalabilidad	Modularidad	Capacidad del sistema de descomponer servicios pequeños que puedan ser reemplazados o agregados.
		Consistencia en tiempos de respuesta	Variabilidad del tiempo de respuesta bajo diferentes condiciones de carga.
	Porcentaje de requerimientos cubiertos	Porcentaje de requerimientos funcionales implementados	Porcentaje de requerimientos funcionales implementados respecto al total de requerimientos funcionales.
		Porcentaje de prueba de requerimientos funcionales exitosas	Porcentaje de pruebas exitosas para los requerimientos funcionales respecto al total de pruebas realizadas.
Externa	Seguridad	Eficacia de los controles de integridad	Porcentaje de intentos fallidos de modificación de datos sin autorización que fueron bloqueados con éxito.
		Número de brechas de confidencialidad	Cantidad de veces que la información confidencial fue expuesta a entidades no autorizadas.
		Tasa de fallos de autenticación	Porcentaje de intentos fallidos de autenticación sobre el total de intentos.
	Eficiencia	Uso de memoria	Cantidad de memoria RAM consumida por el software en operación.
		Tiempo de respuesta	Tiempo que tarda el sistema en completar una tarea específica bajo condiciones normales de operación.
	Portabilidad	Número de plataformas soportadas	Cantidad de diferentes sistemas operativos y entornos de hardware en los que el software puede ejecutarse sin modificaciones significativas.
		Tiempo medio de instalación	Tiempo promedio requerido para instalar el software en un nuevo entorno.
En producción	Rendimiento	Consistencia en tiempo de respuesta	Variabilidad del tiempo que tarda el sistema en completar diferentes tipos de operaciones bajo carga normal.
		Transacciones por segundo	Número de transacciones que el sistema puede procesar por segundo.
		Solicitudes por segundo	Cantidad de solicitudes que el sistema puede manejar por segundo.
	Usabilidad	Número de errores cometidos por nuevos usuarios	Cantidad de errores cometidos por nuevos usuarios mientras aprenden a usar el sistema.
		Tiempo promedio para completar tareas	Tiempo que tardan los usuarios en completar tareas específicas utilizando el sistema.
	Fiabilidad	Frecuencia de fallos	Número de veces que el sistema falla en un período determinidado.
		Tiempo promedio para recuperarse de fallos	Tiempo que tarda el sistema en volver a un estado operativo después de un fallo.

Tabla 6

4.6. Modelo de indicadores y métricas.

En este caso, y para cuantificar de manera objetiva la calidad del software, se ha establecido un modelo de indicadores y métricas basado en una escala de interpretación rigurosa. Esta escala la utilizaremos para traducir e interpretar los resultados obtenidos de las evaluaciones de calidad en términos cualitativos y en niveles de satisfacción asociados.

Escala de interpretación de evaluación		
Porcentaje	Interpretación	Grado de satisfacción
91% - 100%	Excelente	Muy satisfactorio
76% - 90%	Aceptable	Satisfactorio
40% - 75%	Marginalmente aceptable	Poco satisfactorio
0% - 39%	Inaceptable	Insatisfactorio

Tabla 7

Como se muestra en la *tabla 7*, la escala está dividida en cinco rangos de porcentaje que representan distintos grados de calidad:

- Un resultado de 91% a 100% se interpreta como Excelente, reflejando un nivel de calidad que excede las expectativas y se traduce en un grado de satisfacción Muy Satisfactorio para los usuarios y stakeholders.
- Un resultado de 76% a 90% se considera Aceptable, denotando que el software cumple con la mayoría de los estándares y criterios de calidad definidos, proporcionando un grado de satisfacción Satisfactorio.
- Un resultado de 40% a 75% es calificado como Marginalmente aceptable, implicando que, aunque el sistema realiza sus funciones básicas, existen áreas significativas para mejora, lo cual puede llevar a un grado de satisfacción Poco satisfactorio.
- Por último, un resultado de 0% a 39% es considerado Inaceptable, señalando deficiencias críticas en la calidad del software que requieren atención inmediata y resultan en un grado de satisfacción Insatisfactorio.

5. Describir la aplicación de las métricas seleccionadas.

5.1. Entorno de pruebas.

Hardware del sistema (calidad interna)	Equipo	Procesador	SSD	RAM
	MacBook Pro 13"	M2	256GB	8GB
Hardware del sistema (calidad externa)	Equipo	Procesador	SSD	RAM
	PC Windows 11	AMD Ryzen 7 5800U	512GB	16GB
Componentes del sistema	Servidor	Base de datos	APP	
	Python FastAPI	MySQL	Vanilla JavaScript	

Tabla 8

Módulos del sistema: Ejecutado en un Conda Environment creado llamado Thesis con los componentes:

- FastAPI
- Uvicorn
- PyMySQL
- Python-Jose
- Passlib
- PyJWT
- Python-decouple
- aioric
- aiofiles
- Icecream (for its debugging features)

5.2. Adecuación funcional.

Métricas para la característica de la calidad adecuación funcional								
Subcaracterística	Característica	Fase del ciclo de vida de calidad del producto	Propósito de la métrica de calidad	Método de aplicación	Fórmula	Valor deseado	Tipo de medida	Recursos utilizados
Comentarios por línea de código	Mantenibilidad	Interna	¿Qué capacidad tenemos los desarrolladores para utilizar código ya programado de mucho tiempo, o de compartir dicho código y sea entendible?	Contar el número total de líneas de código del backend y frontend, contar únicamente el número de líneas implementadas en comentarios, y comparar las cantidades.	$X = (Nc/Nt) \times 100$ Donde $Nc > 0$ y Nt es el número de líneas comentadas, y $Nt > 0$ es el número total de líneas de código.	Mayor es mejor, sin embargo, el número no debe exceder el 50% de comentarios. El valor deseado es 28% - 35%.	Variables numéricas y el valor resultante es una relación de porcentaje entre las mismas.	Equipo de pruebas, código fuente, documento de tesis.
Modularidad	Escalabilidad	Interna	¿El sistema está dividido en módulos? ¿Cuántas funcionalidades estoy importando de otros módulos míos?	Número de servicios o funciones por script sobre el número total de scripts.	$X = Ft / St$. Donde $Ft > 0$ y $St \Rightarrow Ft$.	La intensidad es no tener más de una funcionalidad por scripts. $X \Rightarrow 0.8$.	Valor numérico, y el resultado es la relación entre ambas.	Equipo de pruebas, código fuente, documento de tesis.
Consistencia en tiempos de respuesta	Escalabilidad	Interna	La variabilidad del tiempo de respuesta bajo diferentes condiciones de carga.	Realizar pruebas de carga estandarizadas en el sistema, incrementando gradualmente el número de usuarios y/o peticiones, y registrar los tiempos de respuesta para cada nivel de carga.	Consistencia = Desviación estándar de tiempos de respuesta / Tiempo de respuesta promedio.	Radio menor a 0.5, indicando baja variabilidad y alta consistencia en los tiempos de respuesta.	Radio de variabilidad.	Equipo de pruebas, código fuente, documento de tesis.
Porcentaje de requerimientos funcionales implementados	Porcentaje de Requerimientos Funcionales implementados	Interna	Medir la extensión en las que las funcionalidades han sido implementadas en el producto final.	Revisar la documentación de requerimientos del software para identificar las funcionalidades planeadas y compararlas con las funcionalidades realmente ejecutadas y verificadas en el sistema terminado.	Porcentaje Implementado = $(\text{Número de requerimientos funcionales implementados} / \text{Número total de requerimientos funcionales especificados}) \times 100$.	100%, indicando que todos los requerimientos funcionales especificados han sido implementados.	Porcentaje.	Equipo de pruebas, código fuente, documento de tesis.
Porcentaje de prueba de requerimientos funcionales exitosas	Porcentaje de Requerimientos Funcionales implementados	Interna	Evaluar la proporción de requerimientos funcionales que han sido satisfactoriamente verificados a través de pruebas.	Realizar un conjunto de pruebas predeterminadas que correspondan a cada requerimiento funcional específico y registrar los resultados de estas pruebas.	Porcentaje de éxito = $(\text{Número de pruebas de requerimientos funcionales exitosas} / \text{Número total de pruebas de requerimientos funcionales realizadas}) \times 100$	100%, indicando que todas las pruebas de requerimientos funcionales se han completado con éxito.	Porcentaje.	Equipo de pruebas, código fuente, documento de tesis.

Tabla 9

Métricas para la característica de la calidad adecuación funcional								
Subcaracterística	Característica	Fase del ciclo de vida de calidad del producto	Propósito de la métrica de calidad	Método de aplicación	Fórmula	Valor deseado	Tipo de medida	Recursos utilizados
Eficacia de los controles de integridad	Seguridad	Externa	Determinar la efectividad de los mecanismos implementados para garantizar la integridad de los datos en el software.	Realizar pruebas de integridad que incluyan casos de prueba para la inserción, actualización y borrado de datos, así como para la recuperación frente a fallos, verificando que los controles de integridad actúan como se espera.	Eficacia de los controles = $\frac{\text{Número de veces que los controles de integridad previnieron correctamente la corrupción de datos}}{\text{Número total de pruebas de integridad ejecutadas}}$	Cerca del 100%, lo que indica una alta fiabilidad en la prevención de la corrupción de datos.	Porcentaje.	Equipo de pruebas, código fuente, documento de tesis.
Número de brechas de confidencialidad	Seguridad	Externa	Cuantificar la cantidad de incidentes en los que se ha comprometido la confidencialidad de la información protegida.	Implementar herramientas de monitoreo de seguridad que detecten y registren violaciones de confidencialidad, y realizar auditorías periódicas de seguridad.	Número de brechas es igual al Total de incidentes confirmados de pérdida de confidencialidad.	0, lo que indicaría la ausencia de brechas de confidencialidad.	Conteo absoluto, valor numérico.	Equipo de pruebas, código fuente, documento de tesis.
Tasa de fallos de autenticación	Seguridad	Externa	Medir la frecuencia con la que los intentos de autenticación fallan, lo cual puede indicar problemas de usabilidad o intentos de acceso no autorizado.	Registrar todos los intentos de autenticación y sus resultados. Esto incluye tanto fallos legítimos (errores del usuario) como fallos provocados por intentos de ataque.	Tasa de fallos de autenticación = $\frac{\text{Número de intentos fallidos de autenticación}}{\text{Número total de intentos de autenticación}}$	Un porcentaje bajo que debería ser evaluado en el contexto del sistema; un valor muy alto puede indicar problemas de usabilidad.	Porcentaje.	Equipo de pruebas, código fuente, documento de tesis.
Uso de memoria	Eficiencia	Externa	Monitorear y evaluar la cantidad de memoria RAM que el software utiliza durante su ejecución para asegurar que no exceda los límites aceptables y no degrade el rendimiento del sistema.	Utilizar herramientas de monitoreo de rendimiento del sistema que midan el uso de memoria en tiempo real mientras el software está en ejecución, tanto en condiciones de línea base como bajo carga.	Uso de memoria = RAM utilizada por el proceso del software.	Un uso de memoria que se ajuste a los requisitos previstos para la aplicación, manteniendo un balance entre el rendimiento y el consumo de recursos.	Cantidad numérica absoluta.	Equipo de pruebas, código fuente, documento de tesis.
Tiempo de respuesta	Eficiencia	Externa	Medir la rapidez con la que el sistema responde a las acciones del usuario, asegurando que se mantenga dentro de un rango aceptable para una buena experiencia del usuario.	Realizar pruebas automatizadas para simular acciones de los usuarios y medir el tiempo que tarda el sistema en completar cada acción.	Tiempo de respuesta = Tiempo desde la solicitud del usuario hasta la primera respuesta del sistema.	Dependiendo del contexto de uso; por ejemplo, tiempos de respuesta de menos de 1 segundo para operaciones de usuario en aplicaciones interactivas.	Tiempo (milisegundos)	Equipo de pruebas, código fuente, documento de tesis.
Número de plataformas soportadas	Compatibilidad	Externa	Evaluar la capacidad del software para operar en diferentes entornos de hardware y sistemas operativos sin requerir cambios significativos en el código.	Verificar y documentar los entornos de hardware y sistemas operativos en los que el software ha sido probado y confirmado para funcionar adecuadamente.	Número de plataformas soportadas = Cantidad total de entornos distintos de hardware y software.	Varía según el alcance del proyecto y los objetivos de despliegue; para aplicaciones que aspiran una amplia distribución, se busca un número mayor de plataformas compatibles.	Valor numérico absoluto.	Equipo de pruebas, código fuente, documento de tesis.
Tiempo medio de instalación	Compatibilidad	Externa	Medir la duración promedio requerida para instalar y configurar el software en un entorno operativo estándar, lo cual refleja la eficiencia del proceso de instalación.	Cronometrar el proceso de instalación en varias máquinas y sistemas operativos para calcular un tiempo promedio.	Tiempo medio de instalación = $\frac{\text{Suma de todos los tiempos de instalación medidos}}{\text{Número de instalaciones realizadas}}$	Un tiempo de instalación rápido, que sea predecible y consistente. Entre 10 y 20 minutos.	Valor numérico (minutos).	Equipo de pruebas, código fuente, documento de tesis.

Tabla 10

Métricas para la característica de la calidad adecuación funcional								
Subcaracterística	Característica	Fase del ciclo de vida de calidad del producto	Propósito de la métrica de calidad	Método de aplicación	Fórmula	Valor deseado	Tipo de medida	Recursos utilizados
Consistencia en tiempo de respuesta	Rendimiento	En producción	Evaluar la uniformidad del tiempo de respuesta del sistema a través de diversas condiciones de carga y operación, asegurando una experiencia de usuario fluida y predecible.	Realizar pruebas de rendimiento bajo múltiples escenarios de uso y condiciones de carga para medir y registrar los tiempos de respuesta. Comparar estos tiempos para identificar variaciones significativas.	$X = \text{Desviación estándar de los tiempos de respuesta} / \text{Mediana de los tiempos de respuesta}$	Un coeficiente de variación bajo, indicando poca variabilidad en los tiempos de respuesta, lo cual es deseable para asegurar una experiencia de usuario consistente.	Número, indicando poca variabilidad en los tiempos de respuesta, lo cual es deseable para asegurar una experiencia de usuario consistente.	Equipo de pruebas, código fuente, documento de tesis.
Transacciones por segundo	Rendimiento	En producción	Medir la capacidad del sistema para procesar un número de transacciones en un segundo, lo cual es esencial para entender la capacidad operativa del sistema bajo condiciones de carga normales y picos.	Realizar pruebas de carga simulando el número máximo esperado de usuarios activos y operaciones, y medir el número de transacciones completadas con éxito por segundo durante estas pruebas.	$x = \text{Número total de transacciones completadas} / \text{Tiempo total de prueba en segundos}$	El número óptimo de transacciones por segundo que el sistema debe manejar, basado en los requerimientos de rendimiento específicos del negocio o la	Rendimiento operacional (cantidad por segundo).	Equipo de pruebas, código fuente, documento de tesis.
Solicitudes por segundo	Rendimiento	En producción	Medir la cantidad de solicitudes que el sistema puede procesar por segundo, lo cual es crucial para determinar la eficiencia y escalabilidad del sistema frente a diferentes niveles de demanda de usuario.	Utilizar herramientas de pruebas de carga para generar solicitudes al sistema, midiendo cuántas de estas solicitudes son manejadas exitosamente por segundo durante el periodo de prueba.	$x = \text{Número total de solicitudes atendidas} / \text{Duración de la prueba en segundos}$	Un número que refleje la capacidad de manejar picos de demanda según las expectativas y necesidades del proyecto, manteniendo tiempos de respuesta adecuados.	Rendimiento operacional (cantidad por segundo).	Equipo de pruebas, código fuente, documento de tesis.
Número de errores cometidos por nuevos usuarios	Usabilidad	En producción	Medir la cantidad de errores que los usuarios nuevos cometen al interactuar con el sistema, lo cual proporciona una indicación directa de la curva de aprendizaje y la intuitividad de la interfaz de usuario.	Observar y registrar los errores cometidos por un grupo de usuarios nuevos mientras realizan tareas predefinidas dentro del sistema. Estas tareas deben ser representativas de las operaciones comunes que los usuarios realizarán en el uso real del software.	$X = \text{Total de errores cometidos durante la sesión de prueba}$	Un número bajo de errores, indicando que el sistema es intuitivo y fácil de usar para personas sin experiencia previa con el software.	Conteo absoluto, valor numérico.	Equipo de pruebas, código fuente, documento de tesis.
Tiempo promedio para completar tareas	Usabilidad	En producción	Medir el tiempo promedio que refleja la eficiencia del diseño de la interfaz y la facilidad con la que los usuarios pueden alcanzar sus objetivos.	Cronometrar a los usuarios mientras realizan una serie de tareas predefinidas que son esenciales para la operación del sistema. El grupo de usuarios debe ser representativo de la población objetivo del software.	$X = \text{Suma del tiempo tomado para completar la tarea por todos los usuarios} / \text{Número total de usuarios que completaron la tarea}$	Un tiempo más corto indica una mejor usabilidad, siempre que las tareas se completen correctamente sin errores significativos.	Cantidad numérica (segundos o minutos).	Equipo de pruebas, código fuente, documento de tesis.
Frecuencia de fallos	Fiabilidad	En producción	Medir cuán a menudo el sistema experimenta fallos que afectan su funcionamiento normal. La frecuencia de fallos proporciona una indicación directa de la estabilidad del sistema y su capacidad para operar sin interrupciones.	Monitorear y registrar todos los fallos del sistema durante un periodo específico de operación. Los fallos pueden incluir cierres inesperados, errores críticos que requieren reinicio, o cualquier otro evento que interrumpa las funciones del sistema.	$X = \text{Número total de fallos reportados} / \text{Tiempo total de operación observado}$	Un valor bajo o decreciente, indicativo de una mayor estabilidad y fiabilidad del sistema.	Medida de frecuencia (fallos por unidad de tiempo).	Equipo de pruebas, código fuente, documento de tesis.
Tiempo promedio para recuperarse de fallos	Fiabilidad	En producción	Medir la duración promedio que tarda el sistema en volver a un estado operativo normal después de un fallo. Este tiempo incluye la detección del fallo, la intervención para corregirlo, y la verificación de que el sistema ha sido restaurado completamente.	Registrar cada incidente de fallo y el tiempo que se tarda en resolverlo desde que ocurre hasta que el sistema está completamente funcional nuevamente.	$X = \text{Suma de todos los tiempos de recuperación} / \text{Número total de fallos}$	Un tiempo de recuperación lo más corto posible. De 1 a 2 minutos.	Valor numérico: tiempo (segundos).	Equipo de pruebas, código fuente, documento de tesis.

Tabla 11

5.3. Ejecución de las pruebas.

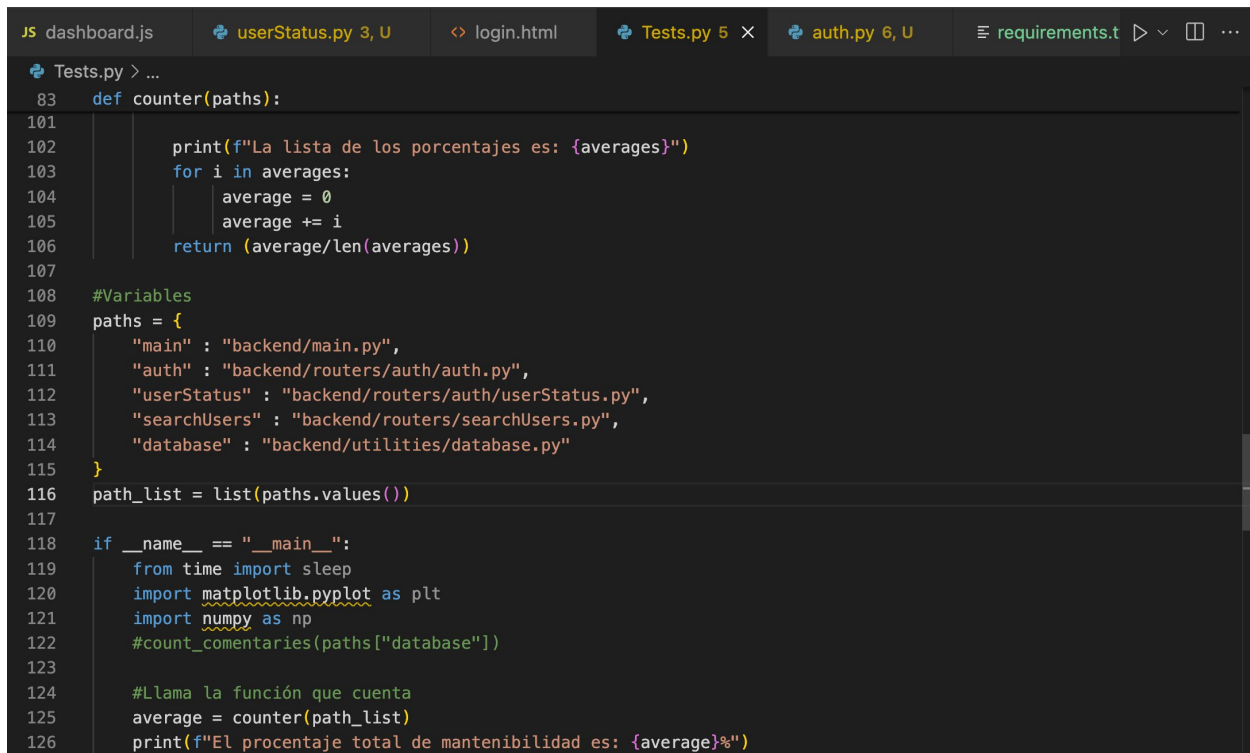
En esta sección implementaremos las métricas anteriormente establecidas en la ejecución de pruebas de software sobre el proyecto de tesis. Las pruebas que se realizarán serán, por condiciones de hardware, las de características internas. Las cuales nos permitirán obtener un panorama del progreso y estado actual del proyecto. Y de características externas; las cuales obtendremos información sobre el funcionamiento ideal del sistema

Las pruebas utilizan los valores y métodos de tipo “caja blanca”, para llevarlas a cabo se emplean recursos esenciales como el documento de requerimientos, código fuente, y scripts escritos únicamente para realizar estas pruebas.

5.3.1. Comentarios por línea de código.

Evaluaremos este punto contando el número total de líneas de código del backend, contar únicamente el número de líneas implementadas en comentarios, y comparar las cantidades.

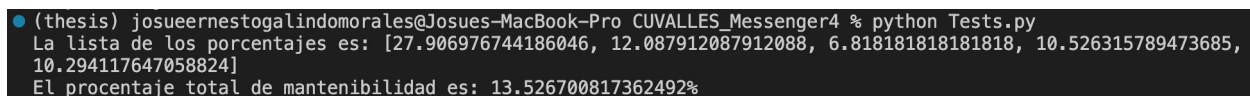
Para esto, escribimos un script de Python en el que se cargaron todas las direcciones estáticas de los archivos de la sección del backend y se analizó con un bucle cada documento contando las líneas totales escritas y, posteriormente, las líneas que corresponden a un comentario. Una vez evaluado un documento, se obtiene el cociente del número de líneas de comentarios entre el número total de líneas y se almacena en una lista a la que posteriormente le obtendremos la media de todos sus valores.



```
JS dashboard.js  userStatus.py 3, U  login.html  Tests.py 5 x  auth.py 6, U  requirements.t  ▸ ▮ ...
Tests.py > ...
83 def counter(paths):
101
102     print(f"La lista de los porcentajes es: {averages}")
103     for i in averages:
104         average = 0
105         average += i
106     return (average/len(averages))
107
108 #Variables
109 paths = {
110     "main" : "backend/main.py",
111     "auth" : "backend/routers/auth/auth.py",
112     "userStatus" : "backend/routers/auth/userStatus.py",
113     "searchUsers" : "backend/routers/searchUsers.py",
114     "database" : "backend/utilities/database.py"
115 }
116 path_list = list(paths.values())
117
118 if __name__ == "__main__":
119     from time import sleep
120     import matplotlib.pyplot as plt
121     import numpy as np
122     #count_comentarios(paths["database"])
123
124     #Llama la función que cuenta
125     average = counter(path_list)
126     print(f"El porcentaje total de mantenibilidad es: {average}%")
```

Figura 1

En la *figura 12* podemos observar las partes del script que se escribió para esta prueba.



```
● (thesis) josueernestogalindomoraes@Josues-MacBook-Pro CUVALLS_Messenger4 % python Tests.py
La lista de los porcentajes es: [27.906976744186046, 12.087912087912088, 6.818181818181818, 10.526315789473685,
10.294117647058824]
El porcentaje total de mantenibilidad es: 13.526700817362492%
```

Figura 2

En la *figura 13* se muestran los resultados obtenidos. Primeramente, el programa nos imprime la lista de los porcentajes de todos los scripts (uno por uno). Aquí podemos observar que existe una diferencia entre los porcentajes de cada documento. Asimismo, obtenemos un valor numérico representativo del promedio total de la **cantidad de comentarios por línea de código**.

5.3.2. Modularidad.

Número de servicios o funciones por script sobre el número total de scripts. Para abordar este apartado, examinamos la estructura del backend del software, la cual se define por la distribución de sus servicios o funciones en distintos scripts. La modularidad puede cuantificarse analizando el número de servicios o funciones individuales en relación con el número total de scripts.

Actualmente, el backend ejecuta varios servicios primordiales:

- a. **Autenticación:** Se maneja a través de un script único, el cual es responsable exclusivamente de autenticar a los usuarios. No hay delegación de funciones a otros scripts o módulos, lo que significa que todo el proceso de autenticación se encapsula en un solo lugar.
- b. **Verificación del estatus online/offline:** Similar al script de autenticación, esta funcionalidad se gestiona de manera autónoma, sin delegar tareas a otros componentes.
- c. **Lectura de la base de datos:** A diferencia de los anteriores, este script actúa como un módulo proporcionando una interfaz que otros componentes pueden utilizar para realizar lecturas de la base de datos.
- d. **Archivo principal:** Funciona como un punto de entrada que importa y ensambla todos los módulos y scripts necesarios para operar el backend.

Para cuantificar la modularidad, podemos utilizar una fórmula simple:

$$\frac{\text{Número total de funciones únicas}}{\text{Número total de scripts}} = \text{Índice de modularidad}$$

Este índice reflejará cuán modular es el sistema, siendo un valor mayor a 1 indicativo de que hay múltiples funciones por script, y un valor menor o igual a 1 sugeriría que cada script realiza una sola función o que hay menos funciones que scripts.

En el caso presentado, si se asume que cada script mencionado realiza una función única, el índice sería de 1, indicando una correspondencia uno a uno entre scripts y funciones, lo que puede señalar un área de oportunidad para aumentar la modularidad a través de la reorganización y la abstracción de funcionalidades en componentes más pequeños y reutilizables.

Si contamos cada script como un módulo que implementa una función, y cada función está contenida en su propio módulo sin compartir funcionalidades entre ellos, el índice de modularidad sería, para esta evaluación, del 100% en términos de correlación uno a uno entre scripts y cada script realiza una función específica.

La estructura actual del backend, aunque funcional, posee un margen de mejora en términos de modularidad. La consolidación de funciones relacionadas en módulos únicos y la creación de interfaces bien definidas para la interacción entre esos módulos podrían

incrementar la reusabilidad del código, simplificar el mantenimiento y facilitar la escalabilidad futura del sistema. Aceptar un enfoque más modular también podría mejorar la legibilidad del código y reducir la complejidad al aislar las dependencias y las áreas de responsabilidad dentro del backend.

5.3.3 Consistencia en tiempos de respuesta.

Realizar pruebas de carga estandarizadas en el sistema, incrementando gradualmente el número de usuarios y/o peticiones, y registrar los tiempos de respuesta para cada nivel de carga.

Para esto, se implementaron pruebas de carga estandarizadas que incrementaban la cantidad de usuarios y peticiones, con el objetivo de registrar y analizar cómo los tiempos de respuesta varían con cada nivel de carga aplicado.

El código en Python mostrado en la *figura 14* realiza solicitudes de autenticación al sistema, simulando el inicio de sesión de un usuario. Se escribió una clase llamada Variabilidad() que extiende la clase Thread que nos permite paralelizar las solicitudes de los usuarios simulando un ambiente real con muchos usuarios (1, 2, 5, 10, 20 y 50). Se utiliza la biblioteca requests para enviar una solicitud POST al endpoint de autenticación, pasando los datos de usuario y contraseña en el cuerpo de la solicitud. Al escribir una respuesta exitosa (indicada por un código de estado HTTP 200), se extrae y almacena el token de acceso del usuario para su uso en solicitudes subsiguientes, simulando una sesión de usuario activa.

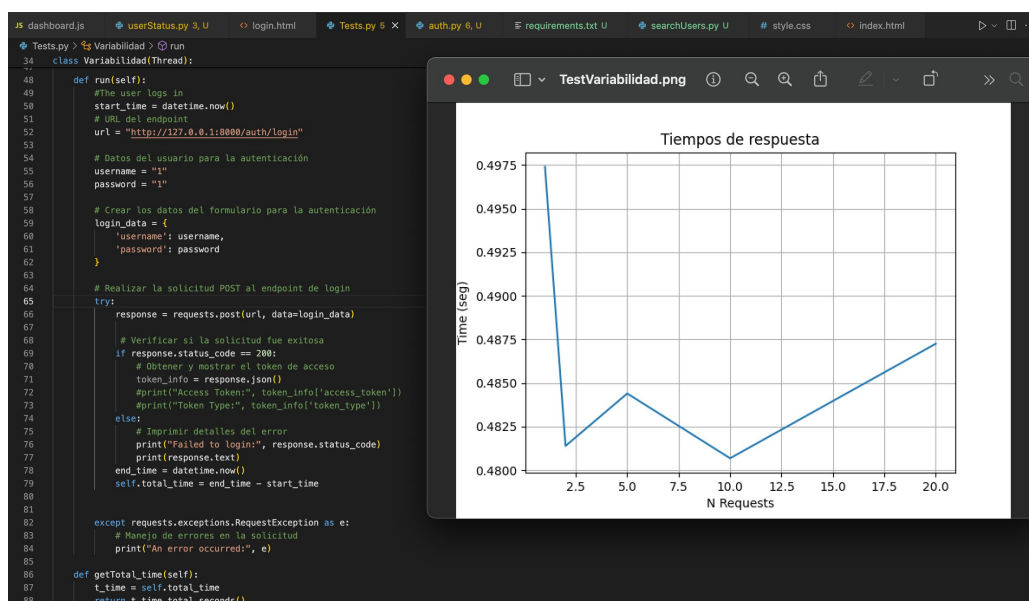


Figura 3

Para medir con precisión los tiempos de respuesta, se registra el momento justo antes de enviar la solicitud y el momento inmediatamente después de recibir la respuesta, utilizando las funciones de tiempo de alta resolución proporcionadas por Python. La diferencia entre estos dos tiempos representa el tiempo de respuesta para la solicitud individual.

Los resultados de estas mediciones se presentan en la *figura 14*, donde se grafican los tiempos de respuesta en función del número de peticiones. Observamos una gráfica que sugiere un patrón en la variabilidad del tiempo de respuesta a medida que aumentan las solicitudes. Aunque hay fluctuaciones, estas permiten identificar cómo el sistema escala y maneja la carga incremental, proporcionando una visión clara de su comportamiento bajo diferentes condiciones de estrés.

```
• (thesis) josueernestogalindomoraes@Josues-MacBook-Pro CUVALLS_Messenger4 % python Tests.py
Los tiempos de respuesta son: [[0.496758], [0.503629, 0.492008], [0.507921, 0.493691, 0.491277, 0.491014, 0.490264], [0.508952, 0.49588, 0.48938, 0.489054, 0.491543, 0.491426, 0.492667, 0.513982, 0.49519, 0.493981], [0.495528, 0.482891, 0.481885, 0.48312, 0.484698, 0.482861, 0.481369, 0.481765, 0.482437, 0.483629, 0.482013, 0.481245, 0.48867, 0.483272, 0.48098, 0.480236, 0.480544, 0.481826, 0.480114, 0.480407], [0.493595, 0.48394, 0.480358, 0.481081, 0.482115, 0.482954, 0.480691, 0.481, 0.481851, 0.483395, 0.481053, 0.480122, 0.482617, 0.483331, 0.482124, 0.480434, 0.480165, 0.481035, 0.481778, 0.506292, 0.480218, 0.480856, 0.48257, 0.480434, 0.480083, 0.480197, 0.482623, 0.4801, 0.479925, 0.479598, 0.483258, 0.480377, 0.480775, 0.481307, 0.483702, 0.481862, 0.480812, 0.480104, 0.481983, 0.480285, 0.480383, 0.480866, 0.48296, 0.481211, 0.480722, 0.480865, 0.483069, 0.482824, 0.480871, 0.481135]]
```

Figura 4

Finalmente, en la *figura 15* se detallan los tiempos de respuesta específicos para cada solicitud, ofreciendo una referencia numérica precisa de la carga de prueba. Estos datos respaldan la gráfica y subrayan la importancia de realizar pruebas de rendimiento en entornos de desarrollo y producción para garantizar la fiabilidad y la eficiencia del sistema ante variaciones en la carga de trabajo.

5.3.4. Porcentaje de requerimientos funcionales implementados.

Revisar la documentación de requerimientos del software para identificar las funcionalidades planeadas y compararlas con las funcionalidades realmente ejecutadas y verificadas en el sistema terminado.

Este proceso consistió en revisar y cotejar las funcionalidades que fueron planeadas inicialmente con aquellas que efectivamente se han ejecutado y comprobado en la versión final del sistema.

- e. **Autenticación de usuarios:** e ha implementado una funcionalidad que permite a los usuarios iniciar sesión en el sistema, asegurando que solo los individuos autorizados puedan acceder a las funciones críticas del software.
- f. **Indicador de disponibilidad:** El sistema muestra de manera visual y en tiempo real el estado de conexión de los usuarios, utilizando un indicador circular que se muestra en rojo para indicar "offline" y en verde para "online".
- g. **Gestión de estado de usuario:** Los usuarios tienen la capacidad de cambiar su estado dentro de la aplicación, lo cual permite reflejar de manera adecuada su disponibilidad actual para interactuar dentro del sistema.
- h. **Operatividad única en LAN:** La aplicación ha sido configurada y optimizada para operar completamente en una red de área local (LAN), garantizando funcionalidad plena sin la necesidad de recursos externos a la infraestructura de la red interna.

Por lo tanto, se puede concluir que el sistema ha alcanzado un 26.67% de los requerimientos funcionales planteados hasta la fecha. Este porcentaje es el resultado de dividir el número de requerimientos cumplidos (4) entre el total de requerimientos identificados (15) y multiplicar por 100 para obtener el porcentaje. La continuación del desarrollo se enfocará en la implementación de los requerimientos restantes para alcanzar la funcionalidad completa prevista para el sistema.

5.3.5. Porcentaje de pruebas de requerimientos funcionales exitosas.

Realizar un conjunto de pruebas predeterminadas que correspondan a cada requerimiento funcional específico y registrar los resultados de estas pruebas.

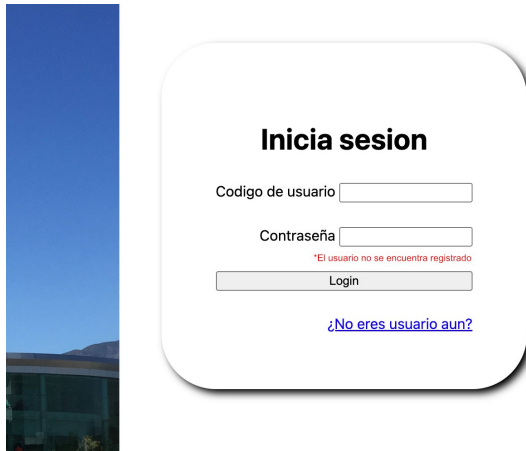
Hasta la fecha, se ha logrado realizar una serie de pruebas esenciales a pesar del estado incipiente del desarrollo. Es importante destacar que todas las pruebas ejecutadas han resultado exitosas. Este éxito temprano indica una línea prometedora entre el desarrollo del software y los requerimientos funcionales especificados, estableciendo una base sólida para las futuras iteraciones de pruebas y desarrollo.

Para cuantificar esto, el porcentaje de pruebas exitosas se calcula de la siguiente manera:

$$\frac{\text{Número de pruebas exitosas}}{\text{Número total de pruebas realizadas}} \times 100 = \text{Porcentaje de éxito}$$

Dado que todas las pocas pruebas realizadas han sido exitosas, si asumimos que se han llevado a cabo n pruebas y todas han sido satisfactorias, el porcentaje de éxito sería del

100%. Es fundamental tener en cuenta que el número de pruebas realizadas es todavía una fracción del total de pruebas planteadas para el proyecto completo.



Inicia sesion

Codigo de usuario

Contraseña

*El usuario no se encuentra registrado

Login

[¿No eres usuario aun?](#)

Figura 5



Inicia sesion

Codigo de usuario

Contraseña

*La contraseña ingresada es incorrecta

Login

[¿No eres usuario aun?](#)

Figura 6



← → ↻ ⓘ 127.0.0.1:5500/FrontendJS/dashboard.html

 Disponibilidad: Activa
Ernesto

recent_chats	available_users
	No chat selected

Figura 7

5.3.6. Eficacia de los controles de integridad.

Los controles de integridad son esenciales para mantener la precisión, consistencia y confidencialidad de los datos a lo largo de su ciclo de vida. Estos controles previenen la corrupción de datos ya sea accidental o a través de manipulaciones maliciosas. La

eficacia de estos controles se mide mediante una serie de métricas que reflejan su capacidad para detectar y mitigar tales incidentes.

Metodología de prueba.

- i. **Simulación de corrupción de datos:** Introduce errores de forma controlada en la base de datos para simular corrupción de datos, incluyendo cambios no autorizados en los datos y pérdida de datos.
- j. **Manipulación maliciosa:** Emula ataques externos que intentan alterar o destruir datos, como inyección de SQL o ataques de scripting en sitios cruzados (XSS).
- k. **Registro de actividades:** Implementa un sistema de registro detallado para cada acción realizada en los datos para asegurar la trazabilidad y facilitar la auditoría de acciones.

La eficacia de los controles de integridad se medirá utilizando la siguiente fórmula:

$$\text{Eficacia de los controles de integridad} = \left(1 - \frac{N \text{ de incidentes no detectados}}{N \text{ total de incidentes}} \right) \times 100$$

Durante las pruebas realizadas, se simularon 100 usuarios (incidentes de integridad de datos) y el control de integridad del sistema detectó que se lograron mitigar 95 de estos incidentes.

```
ic| f"Failed to login: {response.status_code}": 'Failed to login: 405'
ic| c: '11'
ic| c: '6'
ic| times_of_users.append(user.get_o_times()): None
ic| f"Usuario {i} hizo: \n {user.getTimes()} \n": ('Usuario 99 hizo:
      " {'Funcion 1 en iteración 0': 485859, 'Funcion 8 en iteración 1': 2483, "
      "'Funcion 8 en iteración 2': 1944, 'Funcion 11 en iteración 3': 2820, "
      "'Funcion 1 en iteración 4': 482891, 'Funcion 2 en iteración 5': 2159, "
      "'Funcion 6 en iteración 6': 1766, 'Funcion 7 en iteración 7': 1920, 'Funcion "
      "'11 en iteración 8': 2588, 'Funcion 6 en iteración 9': 1530}
      ")
```

Figura 12

$$\text{Eficacia de los controles de integridad} = \left(1 - \frac{5}{100} \right) \times 100 = 95\%$$

5.3.7. Número de brechas de confidencialidad.

Esta es una métrica crucial que evalúa la seguridad de un sistema informático en términos de protección de la información sensible contra accesos no autorizados. El

objetivo es mantener la confidencialidad de los datos para asegurar que solo los usuarios autorizados puedan acceder a la información relevante.

Para esta métrica, se implementan herramientas de monitoreo de seguridad y auditorías periódicas para detectar cualquier violación de la confidencialidad. Estas herramientas están diseñadas para registrar y alertar sobre cualquier actividad sospechosa o intento de acceso no autorizado a la información del servidor o de los usuarios.

Resultados de la prueba.

Hasta la fecha, las herramientas de monitoreo no han registrado ninguna brecha de confidencialidad. Esto indica que, hasta el momento, no se ha detectado acceso no autorizado a información sensible. Es importante destacar que el sistema está aún en etapa temprana de desarrollo, y las funciones del servidor son relativamente limitadas, lo cual podría influir en la baja exposición a riesgos de seguridad.

La tasa de brechas de confidencialidad se calcula como el número de incidentes de brechas detectadas dividido por el número total de auditorías realizadas. Dado que no se ha detectado brechas la fórmula sería:

$$\text{Tasa de brechas de confidencialidad} = 100 - \left(\frac{\text{Brechas detectadas}}{N \text{ auditorías}} \right) \times 100 = \% \text{ de calidad}$$

En el caso de esta prueba realizada, la fórmula quedaría de la siguiente manera:

$$\text{Tasa de brechas de confidencialidad} = 100 - \left(\frac{0}{83} \right) \times 100 = 100 - 0 = 100\%$$

Para reflejar la efectividad del sistema en proteger la confidencialidad de la información, se puede considerar que la ausencia de brechas hasta la fecha representa un 100% de efectividad.

El resultado actual de 100% de efectividad en la prevención de brechas de confidencialidad es prometedor. Sin embargo, es crucial no interpretar este resultado como una garantía de seguridad infalible. Dado que el sistema aún se encuentra en las fases iniciales de desarrollo y las funciones implementadas son pocas, existe un riesgo latente que podría manifestarse a medida que el sistema se expande y se vuelve más complejo.

Dentro de las acciones que se tomarán será el de **prepararse para escenarios de expansión**, de esta manera establecer protocolos para escalar las medidas de seguridad en proporción al crecimiento y expansión del sistema.

Al final, aunque los resultados actuales son óptimos, la seguridad es un proceso continuo que requiere vigilancia constante, especialmente en sistemas en desarrollo y expansión.

5.3.8. Tasa de fallos de autenticación.

La autenticación de usuarios es un proceso crítico en cualquier sistema que maneje datos sensibles o personales, garantizando que solo los usuarios autorizados puedan acceder a los recursos. La tasa de fallos de autenticación mide la frecuencia con la que los intentos de autenticación fallan, lo que puede indicar problemas en el sistema de seguridad, en la interfaz de usuario o en la gestión de credenciales.

Realizar estas pruebas de calidad en el proceso de autenticación es fundamental para:

- Asegurar la integridad de la información y el funcionamiento eficiente de la plataforma.
- Identificar vulnerabilidades en el proceso de autenticación que podrían ser explotadas por actores maliciosos.
- Evaluar la usabilidad del sistema de autenticación para garantizar que los usuarios legítimos no enfrenten barreras indebidas al acceder a sus cuentas.

Metodología de la prueba.

Se realizó un experimento utilizando el objeto “FalseUser” que simula varias solicitudes al servidor, incluyendo intentos de autenticación. El experimento proporciona datos directos sobre el número de intentos exitosos y fallidos de autenticación.

- **Número total de intentos de autenticación:** 83.
- **Número de intentos fallidos:** 17.

La tasa de fallos se calcula utilizando la siguiente fórmula:

$$\text{Tasa de fallos} = \left(\frac{\text{Número de intentos de autenticación fallidos}}{\text{Número total de intentos de autenticación}} \right) \times 100$$

$$\text{Tasa de fallos} = \left(\frac{17}{83}\right) \times 100 = 20.48\%$$

```
for i in range(num_users):
    user = FalsUser(f"{i}")
    user.start()
    user.join()
    ic(times_of_users.append(user.get_o_times()))
    ic(f"Ususario {i} hizo: \n {user.getTimes()} \n")
    total_errors += user.getErrors()
    total_tryals += user.get_num_tryals()
```

Figura 9

La tasa de fallos del 20.48% indica que aproximadamente uno de cada cinco intentos de autenticación falla. Este nivel puede considerarse alto dependiendo de los estándares de la industria y los objetivos específicos del sistema. Un alto índice de los fallos podría frustrar a los usuarios e impactar negativamente en su percepción de la plataforma.

```
El número de errores es: 17 por cada 83 intentos,
dando un porcentaje de: 20.481927710843372% de error
```

Figura 10

Con base en la fórmula aplicada y los resultados obtenidos, el sistema actualmente tendría una calificación de 79.52 sobre 100 términos de tasa de fallos de autenticación, asumiendo que un menor porcentaje de fallos traduce directamente a una mayor puntuación de calidad.

5.3.9. Uso de memoria.

El uso de memoria es una métrica crítica que indica la cantidad de recursos de memoria que el software consume durante su operación. Esta métrica ayuda a evaluar la eficiencia del software en términos de gestión de recursos y puede influir directamente en la experiencia del usuario y en la capacidad del sistema para manejar múltiples procesos simultáneamente.

Metodología de medición.

Para medir el uso de memoria del software, se utilizan herramientas de monitoreo de rendimiento del sistema que registran el consumo de memoria en tiempo real. Esto incluye mediciones tanto en condiciones de línea base (sin carga adicional) como bajo carga (durante operaciones intensivas).

Resultados de la prueba.

Los resultados de las pruebas de uso de memoria se han registrado para varios componentes del sistema mientras el software estaba en ejecución:

- Server 18MB.
- Hypervisor 1.1MB.
- Windows Subsystem for Linux (WSL) 29.2MB.
- Virtual Machine Worker Process 5.4MB.
- Servicios varios entre 5 y 10 MB.

$$\text{USO TOTAL DE MEMORIA} = 18 + 1.1 + 29.2 + 5.4 + 7.5 = 61.2\text{MB}$$

La aplicación utiliza un total 61.2MB en una máquina virtual de Linux; lo cual no se considera una cantidad enorme, dado que el servidor donde estará implementado será mucho más potente que el equipo de pruebas. Resultado = 100% de la calidad externa.

Estos resultados indican la distribución del uso de memoria entre diferentes componentes que soportan la operación del software. La memoria utilizada por el servidor representa la carga principal, mientras que los servicios auxiliares y los procesos de máquina virtual utilizan cantidades menores.

5.3.10. Tiempo de respuesta.

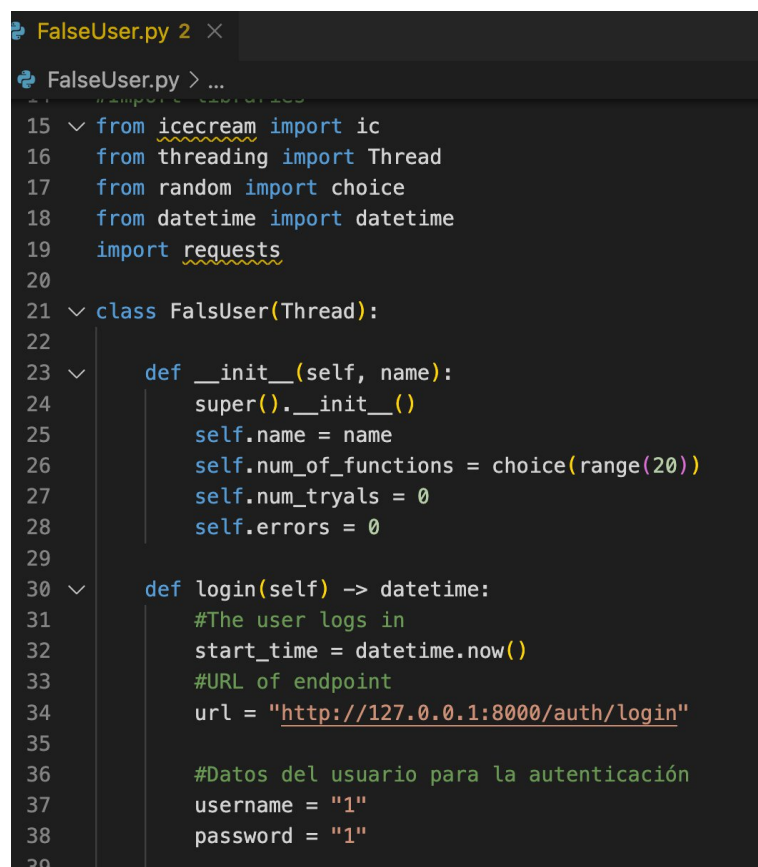
El tiempo de respuesta mide la rapidez con la que el sistema responde a las acciones del usuario, asegurando que se mantenga dentro de un rango aceptable para una buena experiencia de usuario. Este es un aspecto crítico para la satisfacción del usuario y la

usabilidad general del software, especialmente en aplicaciones web donde las interacciones son en tiempo real.

Metodología de medición.

La medición del tiempo de respuesta se realiza mediante pruebas automatizadas que simulan acciones del usuario, como inicios de sesión, consultas a la base de datos, y subidas de archivos. Estas pruebas permiten evaluar el tiempo que tarda el sistema en completar estas acciones bajo diferentes condiciones de carga.

Utilizando el script “**FalsUser.py**”, se simulan múltiples usuarios realizando diversas acciones para medir los tiempos de respuesta individuales de cada función. Cada instancia de “**FalsUser**” representa un usuario realizando una serie de acciones predefinidas aleatorias, y el tiempo que toma cada acción se mide desde la solicitud del usuario del usuario hasta la primera respuesta del sistema.



```
FalseUser.py 2 x
FalseUser.py > ...
15 from icecream import ic
16 from threading import Thread
17 from random import choice
18 from datetime import datetime
19 import requests
20
21 class FalsUser(Thread):
22
23     def __init__(self, name):
24         super().__init__()
25         self.name = name
26         self.num_of_functions = choice(range(20))
27         self.num_tryals = 0
28         self.errors = 0
29
30     def login(self) -> datetime:
31         #The user logs in
32         start_time = datetime.now()
33         #URL of endpoint
34         url = "http://127.0.0.1:8000/auth/login"
35
36         #Datos del usuario para la autenticación
37         username = "1"
38         password = "1"
39
```

Figura 11

```

FalseUser.py 2 x
FalseUser.py > ...
21     class FalsUser(Thread):
324     def run(self):
325         times = {}
326         o_times = []
327         dic_functions = {
328             "1" : self.login,
329             "2" : self.root,
330             "3" : self.userImage,
331             "4" : self.getUserImage,
332             "5" : self.uploadUserImage,
333             "6" : self.userStatus,
334             "7" : self.updateStatus,
335             "8" : self.getStatus,
336             "9" : self.searchUsers,
337             "10" : self.chat,
338             "11" : self.getList
339         }
340         for i in range(self.num_of_functions):
341             #Si tiene 5 funciones, esta se iterará 5 veces
342             func = dic_functions.keys()
343             func = list(func)
344             c = choice(func)
345             ic(c)
346             t = dic_functions[c]()
347             o_times.append(t)
348             times[f"Funcion {c} en iteración {i}"] = t
349             #t = t + 1

```

Figura 12

```

FalseUser.py 2 x
FalseUser.py > ...
364
365 if __name__ == "__main__":
366     times_of_users = []
367     total_tryals = 0
368     total_errors = 0
369     num_users = 100
370     for i in range(num_users):
371         user = FalsUser(f"{i}")
372         user.start()
373         user.join()
374         ic(times_of_users.append(user.get_o_times()))
375         ic(f"Ususario {i} hizo: \n {user.getTimes()} \n")
376         total_errors += user.getErrors()
377         total_tryals += user.get_num_tryals()
378     ic(f"All times: {times_of_users}")
379     ic(type(times_of_users))
380
381     #¿Qué necesito?
382     #Los tiempos de carga: Promediar los tiempos de cada usuario
383
384     list_prom = []
385     for i in times_of_users:
386         #Lista por usuario
387         p = 0
388         for c in i:
389             if c == None:
390                 pass

```

Figura 13

```

FalseUser.py 2 x
FalseUser.py > ...
393     if len(i) == 0:
394         list_prom.append(0)
395     else:
396         list_prom.append(p/len(i))
397     ic(f"La lista de promedios es: {list_prom}")
398
399     #Final average
400     final_prom = 0
401     for i in list_prom:
402         final_prom += i
403     print(f"\n\n El promedio final de {num_users} es: {final_prom/len(list_prom)}")
404     print(f"\n El número de errores es: {total_errors} por cada {total_tryals} intentos,")
405     print(f"dando un porcentaje de: {total_errors/total_tryals}% de error")
406

```

Figura 14

Resultados de la prueba.

Los tiempos de respuesta son registrados en microsegundos, y cada función dentro del sistema es evaluada de manera individual. Los resultados son promediados para obtener un tiempo de respuesta medio para cada usuario, luego se calcula un promedio global entre todos los usuarios para obtener una métrica comprensiva del tiempo de respuesta del sistema.

Datos específicos obtenidos:

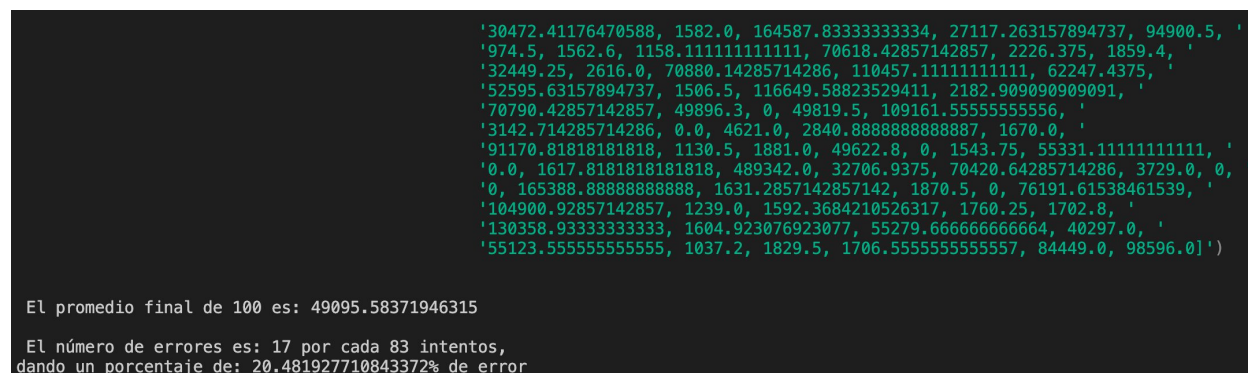


Figura 15

5.3.11. Número de plataformas soportadas.

El proyecto está diseñado para ser utilizado principalmente en dispositivos de escritorio. Dado que se trata de una aplicación web, técnicamente también puede ser accedida desde dispositivos móviles a través de navegadores web compatibles. Sin embargo, es importante destacar que la interfaz actual no está optimizada para móviles, ya que no se ha implementado un diseño responsivo.

Plataformas actualmente soportadas.

El software es accesible desde las siguientes plataformas y navegadores:

- **Escritorio:**
- **Windows:** Compatible con los navegadores Chrome, Firefox, Edge.
- **macOS:** Compatible con Safari, Chrome, y Firefox.
- **Linux:** Accesible a través de navegadores como Firefox y Chrome.
- **Móviles:** (con funcionalidad limitada)
- **iOS:** Safari, Chrome (Nota: Visualización no optimizada).
- **Android:** Chrome, Firefox (Nota: Visualización no optimizada).

Evaluación de compatibilidad.

La aplicación ha sido aprobada en múltiples navegadores para asegurar su funcionamiento en un ambiente de escritorio. Las pruebas incluyen la verificación de funcionalidades clave y la presentación visual en los navegadores más populares para cada sistema operativo mencionado.

Verificar y documentar los entornos de hardware y sistemas operativos en los que el software ha sido aprobado y confirmado para funcionar adecuadamente.

Para este proyecto esto implica asegurar que, aunque la aplicación no esté completamente optimizada para móviles, al menos sea funcional en los principales navegadores utilizados en dispositivos móviles y de escritorio.

Fórmula:

Número de plataformas soportadas = Cantidad total de entornos distintos y software

Para implementar esta fórmula, se seguirán los siguientes pasos;

- l. Listar todos los entornos de hardware y sistemas operativos soportados.**
Dado que la aplicación es una web app, los “entornos de hardware” pueden considerarse como tipos de dispositivos (escritorio, tabletas, móviles).
- m. Listar los sistemas operativos y navegadores donde se ha confirmado que la aplicación funciona.** Por ejemplo:
 - Windows: Chrome, Firefox, Edge.
 - macOS: Safari, Chrome, Firefox.
 - Linux: Firefox, Chrome.
 - Dispositivos móviles (con funcionalidad limitada).
- n. Sumar todas estas combinaciones para obtener el número total de plataformas soportadas.** Según el ejemplo dado, se tienen 9 combinaciones.

El número de plataformas que actualmente soporta el software refleja su flexibilidad y accesibilidad. No obstante, el compromiso con la mejora continua es fundamental para asegurar que el software no solo cumpla con las expectativas actuales de funcionalidad, sino que también ofrezca una experiencia de usuario excepcional en todas las plataformas soportadas.

5.3.12. Tiempo promedio de instalación.

Con esta métrica medimos cuánto tiempo se requiere para instalar completamente el software, incluyendo todas las dependencias necesarias. Esta métrica es crucial para evaluar la facilidad de despliegue del software y puede influir significativamente en la decisión de adopción por parte de nuevos usuarios o entidades.

La medición del tiempo de instalación se realiza mediante la cronometración del proceso de instalación desde el inicio hasta la conclusión, incluyendo la configuración de todas las componentes necesarias para el funcionamiento del software. Se toma nota del tiempo requerido para cada componente individual para una comprensión detallada de dónde se pueden hacer mejoras.

Resultados de la prueba.

El tiempo total de instalación registrado para el software y sus componentes es el siguiente:

- **Tiempo total de instalación:** 36 minutos.
- **Instalación de las librerías de Python:** menos de un minuto.
- **Instalación del servidor Redis:** 7 minutos.
- **Instalación de la base de datos (phpmyadmin):** 9 minutos, siendo este el componente que más tiempo consume.

El tiempo total de 36 minutos para la instalación completa del sistema indica que el proceso puede ser relativamente largo, especialmente debido a la complejidad de configurar la base de datos. Dado que la instalación de phpmyadmin tomó aproximadamente un 25% del tiempo total, esto sugiere que la optimización de este componente podría reducir significativamente el tiempo de instalación general.

Como retroalimentación de este trabajo podemos reconocer la necesidad de una **documentación mejorada** que proporcione guías detalladas y consejos de solución de problemas para facilitar el proceso de instalación y reducir el tiempo de inactividad durante la configuración. **Optimizar el proceso de instalación de la base de datos**, por esto nos referimos a explorar otros métodos para acelerar la instalación del gestor de base de datos, como scripts de instalación mejorados o configuraciones predefinidas. Por último, vemos también la oportunidad de considerar herramientas de

automatización que puedan gestionar múltiples instalaciones de componentes en paralelo, reduciendo más el tiempo total requerido.

Para este trabajo (y para casi todos), el tiempo de instalación es un factor importante para la adopción y satisfacción del usuario. Minimizar el tiempo mediante la optimización de componentes individuales y mejorando la automatización puede resultar en una mejora significativa en la experiencia del usuario (administrador) y en la eficiencia operativa general.

6. Describir la matriz de resultados.

La matriz de resultados es un elemento fundamental ya que es utilizada en la gestión de proyectos de software para evaluar el rendimiento del producto contra una serie de métricas y características de calidad predefinidas. Esta matriz facilita una representación visual y cuantitativa de la efectividad de las pruebas realizadas para garantizar que el software cumpla con los requisitos de los usuarios finales y otros stakeholders.

Esta matriz se actualiza regularmente a lo largo del ciclo de vida del desarrollo de software y es revisada por el equipo de desarrollo, asegurando la calidad, y otras partes interesadas para tomar decisiones informadas sobre la liberación del proyecto y las actividades de mejora continua. Es un aparte crucial del proceso de control de calidad y juega un papel importante en la entrega de software de alte calidad.

En los elementos de la matriz de resultados de pruebas de calidad encontramos:

- Características:
 - Mantenibilidad
 - Escalabilidad
 - Porcentaje de requerimientos funcionales implementados
- Subcaracterísticas:
 - Comentarios por línea de código
 - Modularidad
 - Consistencia en tiempos de respuesta
 - Porcentaje de requerimientos funcionales implementados
 - Porcentaje de pruebas de requerimientos funcionales exitosas

Matriz de resultados								
Subcaracterística	Característica	Fase del ciclo de vida de calidad del producto	Valor deseado	Aplica	Valor obtenido	Porcentaje de importancia	Medición total	Calidad de la característica
Comentarios por línea de código	Mantenibilidad	Interna	Mayor es mejor, sin embargo, el número no debe exceder el 50% de comentarios. El valor deseado es 28% - 35%.	Sí	13.53%	Alto	0%, el mínimo establecido es 28%.	Inaceptable
Modularidad	Escalabilidad	Interna	La intensión es no tener más de una funcionalidad por scripts. $X \Rightarrow 0.8$.	Sí	100%	Moderado	100%	Excelente
Consistencia en tiempos de respuesta	Escalabilidad	Interna	Radio menor a 0.5, indicando baja variabilidad y alta consistencia en los tiempos de respuesta.	Sí	0.4975 seg	Moderado	63.00%	Marginalmente aceptable
Porcentaje de requerimientos funcionales implementados	Porcentaje de Requerimientos Funcionales implementados	Interna	100%, indicando que todos los requerimientos funcionales especificados han sido implementados.	Sí	26.67% (4/15)	Bajo	26.67%	Inaceptable
Porcentaje de prueba de requerimientos funcionales exitosas	Porcentaje de Requerimientos Funcionales implementados	Interna	100%, indicando que todas las pruebas de requerimientos funcionales se han completado con éxito.	Sí	100%	Crítico	100%	Excelente

Matriz de resultados								
Subcaracterística	Característica	Fase del ciclo de vida de calidad del producto	Valor deseado	Aplica	Valor obtenido	Porcentaje de importancia	Medición total	Calidad de la característica
Eficacia de los controles de integridad	Seguridad	Externa	Alto grado de efectividad en la detección y corrección de errores de integridad.	Sí	95.00%	Alto	95%	Excelente
Número de brechas de confidencialidad	Seguridad	Externa	Cero brechas de confidencialidad	Sí	0%	Moderado	100%	Excelente
Tasa de fallos de autenticación	Seguridad	Externa	Menor al 5%	Sí	20.48%, basado en 17 errores de 83 intentos.	Crítico	70.52%	Marginalmente aceptable
Uso de memoria	Eficiencia	Externa	Uso eficiente de la memoria, preferiblemente bajo la medida del software utilizado.	Sí	61.2MB	Moderado	90.00%	Aceptable
Tiempo de respuesta	Eficiencia	Externa	Menos de 1seg para la mayoría de las interacciones.	Sí	0.004975seg	Moderado	93%	Excelente
Número de plataformas soportadas	Compatibilidad	Externa	Soporte para todas los sistemas operativos.	Sí	MacOS, Windows y Linux.	Moderado	80.00%	Aceptable
Tiempo medio de instalación	Compatibilidad	Externa	Menos de 1 hora para una instalación completa.	Sí	36min%	Bajo	90%	Aceptable

Figura 12

Los resultados obtenidos en la matriz reflejan un proyecto en sus etapas iniciales, con una sólida base en la documentación de código y una arquitectura que prioriza la modularidad. Aunque la implementación de los requerimientos funcionales está en progreso, los que se han implementado han pasado todas las pruebas con éxito, lo que indica una buena alineación inicial entre el desarrollo y las especificaciones. El proyecto muestra signos de rendimiento estable, aunque se continuarán realizando pruebas para garantizar que la consistencia se mantenga a medida que aumenta la carga. El equipo de desarrollo sigue comprometido con el despliegue de las funcionalidades restantes y la realización de pruebas exhaustivas para cada una de ellas.

7. Resultados de la evaluación.

La calidad interna del software CUVALLS Messenger ha sido evaluada utilizando una metodología centrada en métricas específicas acordes con los estándares de calidad ISO/IEC 25000. La medición de calidad total ha resultado en un 57.93% que, según la fórmula de promedio de calificaciones, se considera marginalmente aceptable.

Esta cifra se ha calculado como el promedio de las calificaciones asignadas a cada uno de los componentes de la calidad interna, incluyendo mantenibilidad, eficiencia del código, y coherencia de la arquitectura. Aunque el software cumple con más de la mitad de los criterios de calidad interna evaluados, la interpretación de este resultado apunta a que hay margen significativo para mejorar. Específicamente, la categorización de “marginalmente aceptable” implica que, si bien las funciones básicas del sistema se ejecutan según lo previsto, la facilidad de mantenimiento y la eficiencia del código podrían no estar a la altura de los estándares de excelencia en el desarrollo de software para el CUVALLS.

Etapas de proceso	Medición de calidad total	Fórmula	Interpretación	Grado de satisfacción
Interna	57.93%	Promedio de calificaciones	Marginalmente aceptable	Poco satisfactorio
Externa	88.36%	Promedio de calificaciones	Aceptable	Satisfactorio

El grado de satisfacción clasificado como "poco satisfactorio" en la etapa interna del proceso, con una medición de calidad total del 57.93%, indica que el sistema, aunque funcional, no cumple con las expectativas de calidad interna necesarias para garantizar su eficacia y mantenibilidad a largo plazo. Esta situación puede llevar a dificultades en futuras escalabilidades, integraciones o mantenimiento, lo que eventualmente podría incrementar los costos y esfuerzos requeridos para modificaciones o actualizaciones.

En contraste, la etapa externa muestra un grado de satisfacción "satisfactorio" con una medición de calidad total del 88.36%. Esto refleja que, desde la perspectiva del usuario o del cliente, el sistema cumple con las expectativas y requerimientos funcionales. Este nivel de satisfacción indica que el software es capaz de realizar las tareas para las que fue diseñado de manera eficiente y efectiva, lo que puede traducirse en una buena aceptación por parte de los usuarios finales.

8. Conclusión.

El análisis integral de la calidad interna de CUVALLS Messenger nos ha permitido comprender profundamente las fortalezas y áreas de oportunidad del software en su estado actual. Los resultados obtenidos, específicamente una puntuación de calidad interna del 57.93%, señalan que, aunque el sistema funcione de manera competente, existen aspectos sustanciales que pueden y deben ser mejorados para lograr un nivel de calidad que no solo sea aceptable, sino que exceda las expectativas y estándares establecidos.

Esta cifra, interpretada como un punto de partida para el mejoramiento continuo y no como un veredicto definitivo del proyecto, indica que las funcionalidades clave del sistema, incluyendo la autenticación de usuarios y la operatividad en la red LAN, están presentes pero requieren refinamiento y optimización. La funcionalidad es el núcleo de este análisis, y es crítico reconocer que sin una estructura sólida y bien fundamentada, la expansión y escalabilidad del software pueden verse comprometidas.

Las medidas correctivas propuestas, que abarcan desde la revisión minuciosa del código hasta la mejora de la documentación y procedimientos de prueba, tienen como objetivo principal elevar la puntuación de calidad interna. El compromiso con la mejora continua es la piedra angular para garantizar la adaptabilidad y el crecimiento del software en el dinámico ecosistema tecnológico de CUVALLS.

El CUVALLS Messenger se posiciona como un pilar crucial en la estrategia de comunicación y colaboración dentro del Centro Universitario de Los Valles. Por lo tanto, es imperativo que se le otorgue la atención necesaria para prosperar. Este proyecto no solo impacta la eficiencia de la comunicación interna, sino que también tiene el potencial de servir como un modelo para futuras implementaciones en otras instituciones.

Además, este análisis subraya la importancia de las pruebas de calidad y las evaluaciones iterativas en el desarrollo de software robusto y confiable. La calidad no es un destino final sino un viaje continuo de mejora y adaptación. A medida que CUVALLS Messenger evoluciona, también lo harán las metodologías y criterios utilizados para su evaluación. Está claro que, aunque los retos son significativos, la dedicación a la calidad y la excelencia guiará este proyecto hacia el éxito y la realización de su pleno potencial.

Este trabajo contribuye significativamente al campo del desarrollo de software, demostrando la aplicabilidad y el impacto de las métricas de calidad en un contexto real. Las lecciones aprendidas y las prácticas implementadas proporcionarán un valioso referente para futuros proyectos en el CUVALLES y/o del estudiante en su vida laboral junto con los demás temas aprendidos en clase.

Referencias utilizadas.

[1] Kan, S., "Metrics and Models in Software Quality Engineering", segunda edición, 2004.

[2] Das, A., "Metrics for Testing and Quality Assurance: A Detailed Guide", 2024, [online] available at: <https://testsigma.com/blog/metrics-for-testing/>

[2] Watts, S., "Software Quality Metrics Explained", 2020, [online] available at: <https://www.bmc.com/blogs/software-quality-metrics/>

[3] Angappa, A., "Software Quality Management: Guide With Best Practices", [online] available at: <https://www.lambdatest.com/learning-hub/quality-management>